Guidance to Requirements: Knowledge-Based Trustworthy AI Compliance

Anonymous submission

Abstract

Ensuring compliance with Trustworthy AI guidelines remains a significant challenge due to the complex, socio-technical language used in policy documents. Existing approaches often support extraction or summarization, but they do not fully address how new regulatory knowledge can be compared, validated, and integrated into an evolving knowledge base. This paper presents a human-in-the-loop knowledge engineering framework for converting unstructured trustworthy AI documents into structured knowledge graph representations. The framework combines semantic filtering, novelty assessment, LLM-based triplet extraction, and provenance-aware graph updates. The main contribution is a graph-guided pipeline that filters and integrates regulatory knowledge in relation to existing knowledge graph context, rather than treating each document in isolation. We evaluate the prototype through end-to-end extraction scenarios using representative Trustworthy AI documents. The results show that the framework can identify relevant statements, distinguish new from existing knowledge, and support controlled knowledge graph expansion. This work provides a foundation for future compliance reasoning systems that can connect trustworthy AI requirements with actionable mitigation strategies.

1 Introduction

As AI regulations like the EU AI Act and NIST Risk Management Framework emerge, organizations struggle to translate “legal-speak” into technical requirements. The “headache” for users lies in the ambiguity of terms used in policy documents.

In this work, we propose a "closed-loop" system. We contribute: (1) An NLP-driven extraction pipeline for TAI requirements; (2) A method for expanding existing knowledge bases using LLMs; and (3) A mapping mechanism that connects extracted risks to concrete mitigation strategies.

Artificial Intelligence (AI) systems are increasingly being deployed in high-impact domains such as healthcare, finance, transportation, education, and public administration. As the influence of these systems continues to grow, concerns related to transparency, fairness, accountability, privacy, robustness, and human oversight have become central topics in AI governance. In response, governments and organizations have introduced a growing number of regulatory and governance frameworks aimed at promoting Trustworthy AI, including the European Union AI Act, the NIST AI Risk Management Framework, and the OECD AI Principles.

Although these frameworks provide important guidance, they are typically written in broad and socio-technical language. As a result, practitioners often face difficulties translating high-level regulatory requirements into concrete engineering actions during AI system development. Requirements related to explainability, accountability, or risk management are usually described in natural language and lack direct operational mappings to technical workflows, tools, or mitigation strategies.

Existing approaches in legal NLP and compliance engineering mainly focus on isolated tasks such as document summarization, keyword extraction, policy retrieval, or requirement extraction. While these approaches are useful, they often treat documents independently and do not fully address how extracted knowledge can be compared with existing knowledge, validated, refined, and integrated into an evolving compliance knowledge base. Similarly, static knowledge graphs and retrieval systems often lack mechanisms for contextual filtering, novelty assessment, provenance tracking, and human oversight.

These limitations highlight the need for structured and continuously expandable knowledge systems that can support the operationalization of Trustworthy AI requirements. Such systems should not only extract knowledge from regulatory documents, but also contextualize it relative to existing knowledge, identify redundant or novel information, and support reliable human-supervised integration into machine-readable representations.

To address this challenge, this paper presents a human-in-the-loop knowledge engineering framework for transforming unstructured trustworthy AI documents into structured knowledge graph representations. The proposed framework combines keyword-guided graph retrieval, semantic filtering, novelty assessment, LLM-based triplet extraction, and provenance-aware graph updates within a continuously expandable compliance reasoning pipeline. Unlike conventional extraction pipelines, the proposed approach conditions the extraction process on existing knowledge graph context, allowing new information to be evaluated relative to already known concepts before integration.

The framework is implemented as an end-to-end prototype that supports iterative knowledge graph refinement from trustworthy AI and regulatory documents. The system is designed not only to support structured compliance knowledge engineering, but also to provide a foundation for future mitigation-oriented AI governance systems capable of linking extracted requirements to actionable technical recommendations.

The research presented in this paper is guided by the following research questions:

1. How can unstructured trustworthy AI regulatory documents be transformed into structured and machine-actionable knowledge representations?
2. How can semantic filtering and novelty assessment reduce noise and redundancy during knowledge graph expansion?
3. How can human oversight be integrated into LLM-driven regulatory knowledge extraction pipelines to improve trustworthiness and reduce hallucinated compliance knowledge?
4. Can extracted trustworthy AI requirements be linked to actionable technical mitigation strategies?

This paper makes the following contributions:

1. A multi-stage knowledge engineering pipeline for transforming Trustworthy AI-related documents into structured and expandable knowledge graph representations.
2. A graph-guided semantic filtering mechanism that contextualizes extracted candidate statements relative to existing graph relations before integration.
3. A novelty-aware classification approach for distinguishing between existing, partially new, and novel regulatory knowledge.
4. A human-in-the-loop compliance extraction workflow that supports reliable and auditable knowledge graph refinement.

2 Background and Related Work

2.1 Trustworthy AI Governance

The rapid adoption of AI systems in high-impact domains has led to increasing attention toward AI governance and regulatory compliance. Several international initiatives have emerged to define principles and operational requirements for Trustworthy AI. Among the most influential is the European Union Artificial Intelligence Act (EU AI Act), which introduces a risk-based regulatory framework for AI systems and establishes obligations related to transparency, accountability, human oversight, robustness, and data governance.(European Parliament and Council of the European Union 2024; Ho-Dac 2024)

In parallel, the National Institute of Standards and Technology (NIST) introduced the AI Risk Management Framework (AI RMF), which provides guidance for identifying, assessing, and managing AI-related risks throughout the system lifecycle. The framework emphasizes governance, mapping, measurement, and risk management practices for trustworthy AI deployment.(Tabassi 2023)

The OECD AI Principles represent one of the earliest intergovernmental efforts toward AI governance. The principles promote AI systems that are transparent, accountable, robust, fair, and aligned with democratic values and human rights.(Organisation for Economic Co-operation and Development 2019)

More recently, ISO/IEC standards such as ISO/IEC 42001 have introduced management-system-oriented approaches for AI governance and compliance. These standards aim to operationalize governance requirements into organizational processes and auditable controls.(International Organization for Standardization 2023)

Although these frameworks provide important governance foundations, they are primarily expressed in high-level socio-technical language. Several studies note that organizations continue to struggle with translating ethical and regulatory principles into concrete engineering actions and operational workflows.(Jobin, Ienca, and Vayena 2019; Mittelstadt 2019; Hagendorff 2020)

2.2 Knowledge Graphs for AI Governance

Knowledge graphs have increasingly been explored as mechanisms for representing structured and explainable knowledge in AI governance and compliance systems. Their ability to model entities, relationships, provenance, and semantic dependencies makes them particularly suitable for governance-oriented reasoning tasks.(Hogan et al. 2021; Lebo et al. 2013)

Recent work highlights the growing role of knowledge graphs in supporting AI governance, semantic compliance analysis, and trustworthy AI engineering. Meroño-Peñuela et al. propose KG.gov, a framework positioning knowledge graphs as a backbone for AI data governance and lifecycle management.(Meroño-Peñuela et al. 2025) Similarly, foundational knowledge-graph and provenance standards emphasize graph-based representations for human-centered and auditable AI systems.(Hogan et al. 2021; Lebo et al. 2013)

Legal and policy-oriented knowledge graphs have also received increasing attention. These systems aim to transform legal or regulatory documents into machine-readable representations that support querying, reasoning, and compliance analysis. Recent studies on knowledge graph representations for policy reasoning suggest that graph-augmented systems can improve contextual understanding and compliance-oriented reasoning compared to purely text-based retrieval approaches.(Baldwin and Ghanavati 2026)

Within policy engineering, graph-based representations are particularly valuable because regulations often contain interconnected obligations, dependencies, exceptions, and references that are difficult to capture through flat document representations alone. Knowledge graphs therefore provide an important foundation for structured compliance reasoning and provenance-aware governance systems.

2.3 LLM-Based Requirement Extraction

Recent advances in Large Language Models (LLMs) have significantly influenced research in legal NLP, requirement engineering, and policy extraction. Transformer-based models are increasingly used for extracting structured requirements from complex legal and regulatory documents.(Chalkidis et al. 2020)

Several works have explored LLM-assisted regulatory requirement extraction using retrieval-augmented generation (RAG) techniques. Abualhaija et al. proposed XTRAREG, an LLM+RAG framework for extracting regulatory requirements from GDPR-related documents, demonstrating strong extraction performance on privacy-related requirements.(Lewis et al. 2020; Abualhaija et al. 2025)

RAG-based approaches have also become increasingly important in legal AI systems because they improve grounding and contextual reasoning by retrieving domain-specific evidence before generation. Studies in legal RAG demonstrate that retrieval-based augmentation can improve answer quality, traceability, and contextual relevance in regulatory reasoning tasks.(Lewis et al. 2020; Pipitone and Alami 2024)

More recent work such as De Jure proposes iterative LLM self-refinement pipelines for extracting structured regulatory rules from raw legal documents. These systems demonstrate how LLMs can support structured compliance extraction and rule generation across multiple domains including finance, healthcare, and AI governance. (Guliani et al. 2026)

Despite these advances, many existing approaches primarily focus on extraction quality and downstream question answering. They often do not address how extracted knowledge should be contextualized relative to existing knowledge structures, how redundancy should be controlled during graph expansion, or how human oversight should be integrated into evolving compliance knowledge systems.

2.4 Human-in-the-Loop AI Governance

Human oversight is widely recognized as a central requirement in Trustworthy AI governance. Regulatory frameworks such as the EU AI Act explicitly emphasize the need for meaningful human oversight in high-risk AI systems.(European Parliament and Council of the European Union 2024; Ho-Dac 2024)

Within AI governance research, human-in-the-loop (HITL) approaches are commonly proposed to improve reliability, accountability, auditing, and alignment. Human oversight becomes particularly important in legal and regulatory contexts where fully automated interpretation may introduce hallucinations, incorrect reasoning, or misleading compliance conclusions.

Recent discussions in AI governance increasingly highlight the importance of auditable and expert-supervised workflows. Governance literature argues that compliance systems should support transparency, provenance tracking, and expert validation rather than relying entirely on autonomous decision-making.(Tabassi 2023; International Organization for Standardization 2023; Lebo et al. 2013)

In legal NLP and compliance engineering, HITL workflows are often used to validate extracted requirements, review generated outputs, and correct ambiguous interpretations. Such approaches are particularly important in sociotechnical governance domains where legal meaning and governance interpretation are context dependent and cannot be fully reduced to automated rule extraction.

2.5 Gap Analysis

Existing research demonstrates significant progress in AI governance, legal NLP, knowledge graph construction, and regulatory requirement extraction. However, most existing systems address only isolated parts of the compliance operationalization pipeline.

Many approaches focus on requirement extraction or policy summarization without integrating graph-based contextual reasoning. Other systems rely on static compliance databases or retrieval pipelines that do not support iterative knowledge refinement. Similarly, current extraction systems often lack novelty assessment mechanisms for distinguishing redundant from genuinely new regulatory knowledge.

Existing work also rarely combines provenance-aware graph insertion, contextual graph-guided filtering, and human-in-the-loop governance within a continuously expandable knowledge engineering workflow. Furthermore, most current approaches stop at extraction and retrieval, without exploring how extracted regulatory knowledge may later support mitigation-oriented AI governance and operational compliance assistance.

The framework proposed in this paper attempts to unify these aspects within a single pipeline. Specifically, it combines semantic retrieval, subgraph-conditioned filtering, novelty-aware graph refinement, provenance-aware triplet insertion, and human-supervised governance into an iterative knowledge engineering framework for Trustworthy AI compliance.

2.6 LLMs in Legal and Ethical Domain

Discussion of how transformer-based models have been used for requirement engineering and the limitations of current approaches.

3 Framework Overview

This paper proposes a HITL-supported knowledge engineering framework for operationalizing Trustworthy AI requirements from unstructured regulatory and guideline documents. As shown in Figure 1, the framework follows a closed-loop process in which extracted knowledge is validated, inserted into a knowledge graph, and reused as context in future iterations.

The pipeline starts by retrieving existing graph relations related to a selected Trustworthy AI topic, such as explainability, fairness, or accountability. These relations provide the semantic context for processing new documents. Candidate statements are then extracted from the uploaded corpus and filtered against the retrieved graph context. This graph-conditioned filtering helps reduce irrelevant information and ensures that new knowledge is interpreted in relation to the existing knowledge base.

The framework then applies novelty assessment to classify candidate statements as existing, partially new, or new knowledge. This supports controlled graph expansion and reduces redundant insertion. Before integration, a human reviewer validates the extracted statements and can reject or refine uncertain outputs.

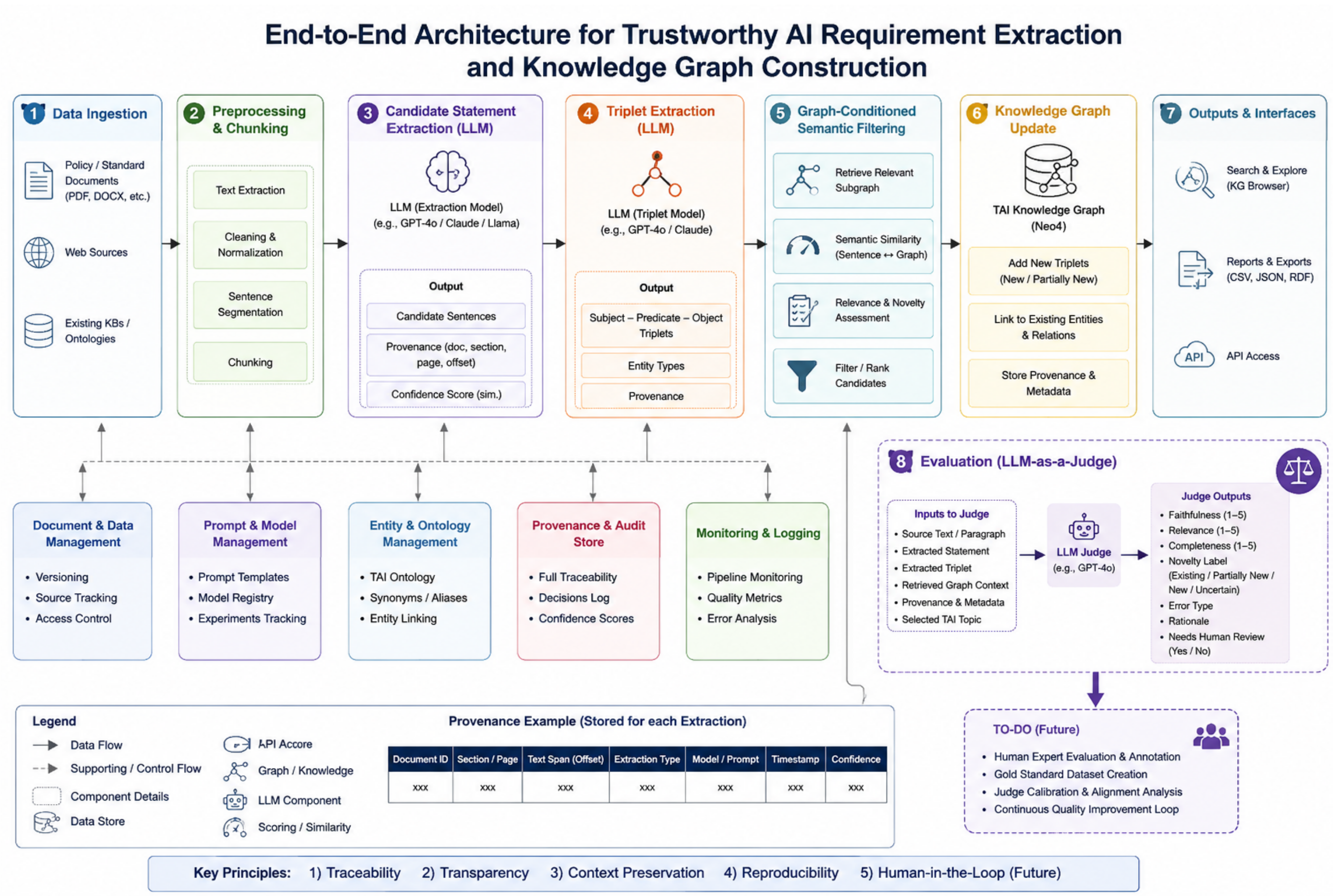

Figure 1: Preliminary end-to-end architecture of the proposed human-in-the-loop knowledge engineering framework for Trustworthy AI requirement extraction and knowledge graph refinement. The framework retrieves existing graph context for a selected Trustworthy AI topic, processes regulatory documents into candidate statements, applies graph-conditioned semantic filtering and novelty assessment, validates uncertain outputs through human oversight, and converts accepted statements into subject–predicate–object triplets. The validated triplets are inserted into the knowledge graph with provenance metadata, enabling iterative and auditable knowledge refinement. The LLM-as-a-judge module is currently included as a provisional evaluation component for assessing faithfulness, relevance, completeness, and novelty. The architecture is AI-generated and subject to further refinement.

Accepted statements are converted into subject–predicate–object triplets and inserted into the knowledge graph with provenance metadata. The updated graph then becomes the basis for future extraction cycles, enabling iterative and auditable refinement of Trustworthy AI compliance knowledge.

4 Methodology

This section describes the updated technical methodology used by the prototype. The architecture figure in Section 3 remains the high-level view of the proposed framework. The diagrams in this section provide a more detailed implementation view of how the system ingests documents, filters candidate requirements, compares them with existing graph context, validates extracted triplets, and persists provenance-aware graph updates.

4.1 Technical Architecture

The implementation follows a four-layer architecture. The browser layer provides the interactive interface, including the Cytoscape graph view and task-specific tabs for ingestion, review, and comparison. The Flask layer exposes route blueprints and maintains a background job store. This design keeps long-running operations asynchronous: expensive tasks are launched as jobs and the user interface polls their status rather than blocking.

The core layer implements the ingestion pipeline and comparison engine. It coordinates document parsing, keyword filtering, graph-context retrieval, semantic filtering, novelty assessment, LLM-based extraction, deterministic validation, and Neo4j upsertion. The external-services layer contains the LLM backend, Neo4j database, sentence-transformers encoder, caches, and logs. Separating these layers makes the pipeline easier to audit because each stage has explicit inputs, outputs, and persistence boundaries.

4.2 End-to-End Ingestion Pipeline

The ingestion workflow is organized as an eleven-stage pipeline. It begins with document upload and parsing, applies a keyword gate to identify topic-relevant paragraph chunks, retrieves graph context from Neo4j, decomposes retained paragraphs into candidate requirement statements, filters those statements against graph context, assesses novelty, routes uncertain outputs through human oversight, extracts triplets, validates them against the standard schema, supports triplet review, and finally writes accepted relations to Neo4j with provenance metadata.

The pipeline is intentionally organized around load-bearing control points. The first control point is the Neo4j subgraph read, which provides the context used by later filters. The second is the human oversight gate, which prevents uncertain candidate statements from flowing directly into extraction. The third is deterministic schema validation, which checks whether generated triplets are source-supported and schema-compatible. The final control point is append-only graph upsertion, which updates graph knowledge without erasing previous provenance records.

4.3 Keyword Gate and Corpus Filtering

The keyword gate is a binary relevance filter rather than a ranker. A paragraph chunk is retained as soon as it satisfies any tier in a four-tier cascade. Tier 1 performs a literal whole-word keyword match using a word-boundary regular expression. Tier 2 performs the same literal scan over an expanded synonym list. The synonym list is assembled from curated defaults, a runtime cache, and an LLM-generated expansion only when a cache miss occurs.

Only paragraphs that fail the first two tiers are passed to model-based checks. Tier 3 uses KeyBERT over an all-MiniLM-L6-v2 encoder to extract the top eight single-word keyphrases from the paragraph. If the selected keyword or one of its synonyms appears among these keyphrases, the paragraph is retained. Tier 4 applies semantic fallback checks with cosine similarity: the keyword embedding is compared with the whole-paragraph embedding using a threshold of 0.45, and unresolved cases compare the keyword embedding with each extracted keyphrase using a stricter threshold of 0.65. Paragraphs that clear none of these tiers are marked unmatched and do not reach decomposition.

The different thresholds reflect the different information density of the compared text. A whole paragraph may contain surrounding context that dilutes the embedding, so the paragraph-level threshold is lower. A keyphrase is more focused, so it must match more strongly. These thresholds, together with the synonym list, are the main tuning levers for balancing recall and precision at the corpus-filtering stage.

4.4 Graph-Conditioned Similarity Filter

After paragraph filtering and candidate-statement decomposition, the pipeline applies a graph-conditioned similarity filter. This stage asks whether each candidate statement discusses concepts that are represented in the retrieved knowledge graph subgraph. The default mode is node-entity matching rather than full-sentence matching. KeyBERT extracts up to eight entity-like keyphrases from each candidate statement using keyphrases of up to three tokens. These entities are then embedded with MiniLM and searched against a node-label index built from the retrieved subgraph.

The decision is based on the maximum similarity between all extracted entities and their nearest graph nodes. In the current prototype, each entity retrieves its top five nearest nodes, and the candidate statement is retained if the best score is at least 0.55. If no entity matches a graph node above the threshold, the candidate is dropped or archived. Retained candidates carry their matched node neighbors forward, and the relations around these neighbors become the comparison context used by novelty assessment.

This design differs from the earlier keyword gate. The keyword gate determines whether a paragraph is broadly relevant to the selected topic; the graph-conditioned filter determines whether a decomposed candidate statement is connected to concepts already present in the graph. A sentence-mode alternative can compare full candidate embeddings against graph-relation sentences, but node-entity matching is preferred because it provides more explicit neighbor context for downstream novelty decisions.

4.5 Novelty Assessment

Novelty assessment is an LLM-based semantic judgment over each retained candidate statement and the neighbor sentences produced by the graph-conditioned filter. The comparator labels each candidate as `EXISTING`, `PARTIALLY_NEW`, or `NEW`. An `EXISTING` label means the candidate is a rephrasing of an existing neighbor and introduces no new constraint or concept. A `PARTIALLY_NEW` label means the candidate overlaps with existing graph knowledge but adds a meaningful concept, condition, scope, modality, or relation. A `NEW` label means the candidate is mostly not covered by the retrieved neighbors.

Cosine similarity is treated only as a weak signal at this stage. A high similarity score can nudge the decision when the candidate and neighbor have the same meaning, but content is decisive. The comparator is run deterministically with temperature zero and JSON output so that novelty labels are stable enough to drive downstream review and extraction. Importantly, all three novelty labels can flow forward. Even an `EXISTING` statement from a new document may be useful as cross-document overlap evidence, so its provenance can still be recorded.

4.6 Triplet Extraction and Schema Validation

Triplet extraction has two phases. In the first phase, an LLM proposes subject–predicate–object triplets for each accepted candidate statement. The model is instructed to copy subjects and objects from the source sentence, use predicates from a flat allowed predicate list where possible, and include modality information when the statement expresses normative force such as requirements or recommendations. The current implementation does not yet implement a predicate-family abstraction. That is, predicates are not grouped into higher-level families such as normative, causal, evidential, alignment, or

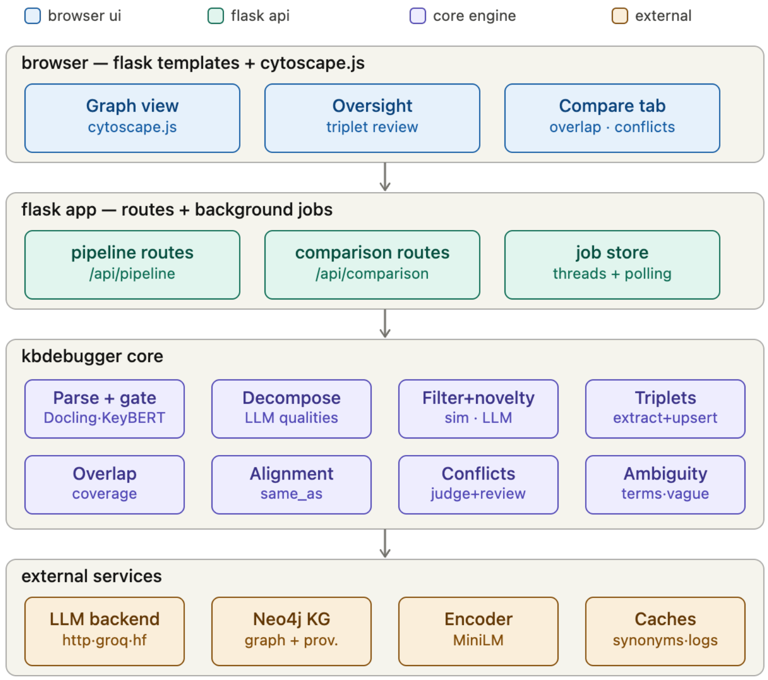

Figure 2: Updated four-layer technical architecture of the prototype. The browser layer exposes the interactive graph and review interface; Flask manages route blueprints and background jobs; the core layer executes ingestion and comparison logic; and external services provide LLM, Neo4j, embedding, cache, and logging functionality.

conflict predicates during extraction. Predicate-family design is treated as planned schema work rather than as a completed component of the reported system.

In the second phase, deterministic validation decides each triplet's status. This validator is the gatekeeper: it checks whether the subject and object are supported by the source statement and whether the predicate belongs to the allowed predicate list or is otherwise retained as a non-standard predicate for review. Triplets labeled `SCHEMA_VALID` are included by default when the entities are source-supported and the predicate is accepted. Triplets labeled `NEEDS_SCHEMA_REVIEW` contain plausible predicates but uncertain entity typing or inferred schema objects. Triplets labeled `NO_SCHEMA_FIT` fail source-support or schema checks. Triplets labeled `NON_STANDARD_PREDICATE` use a predicate outside the allowed list; they are retained and tagged for review rather than silently discarded.

The main methodological principle is that the LLM proposes and deterministic code disposes. Faithfulness is not accepted solely because the model produced a fluent triplet; it is checked against the source statement. This makes the output more auditable and provides a natural target for future multi-agent criticism, where an additional LLM critic can focus only on borderline validation statuses rather than every extracted triplet. A future extension will add predicate-family validation so that individual predicates can be checked not only as surface labels, but also as members of semantically constrained predicate groups.

4.7 Graph Upsertion and Provenance

Validated triplets are inserted into Neo4j through an append-only upsert procedure. Concepts are represented as graph nodes and typed relationships connect them. Each relationship stores a `provenance_records` array rather than a single source field. When the same triple is supported by multiple standards or documents, the system appends the new source record without overwriting the existing evidence. This design preserves cross-document overlap and supports later auditing.

The graph model also supports overlays for alignment and conflict. Alignment records can mark concepts as `same_as` or `not_same_as`, while conflict nodes can connect disputed concepts through explicit involvement relations. These overlays are kept separate from the base regulatory triples so that comparison findings can be reviewed and revised without corrupting the underlying knowledge graph.

4.8 Comparison Engine

The comparison engine operates after ingestion and reads the provenance layer rather than re-ingesting the source documents. It follows a funnel design. Cheap graph queries first identify candidate overlaps, divergences, and conflicts from stored provenance and graph structure. The LLM is used only after this narrowing step, where it judges already-found candidate pairs. Human confirmation then determines which findings are persisted back into the graph as alignment or conflict overlays.

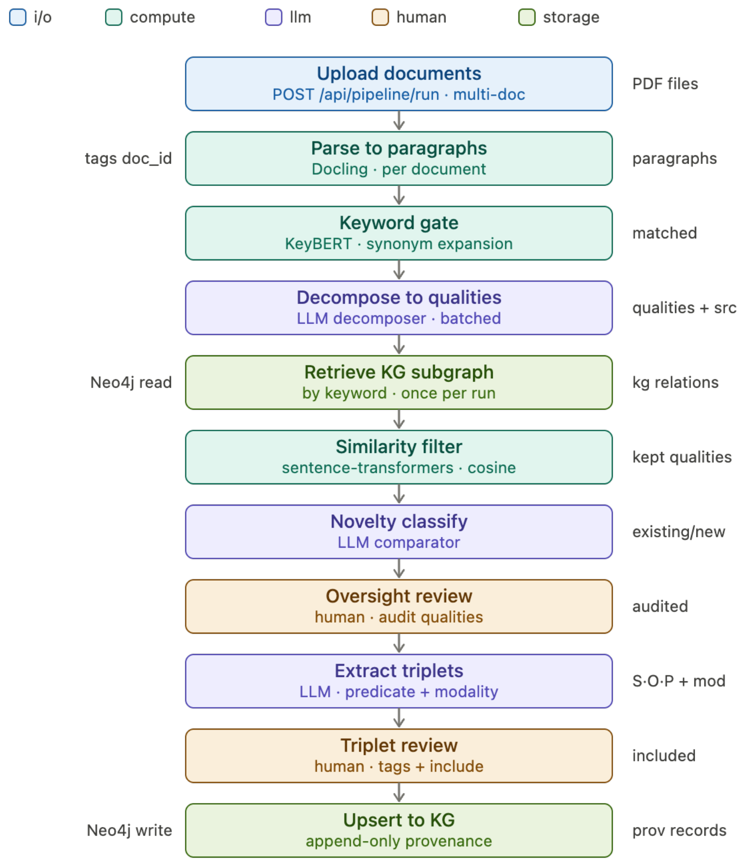

Figure 3: Updated ingestion pipeline. The stages are color-coded by responsibility: input/output operations, deterministic computation, LLM reasoning, human review, and graph storage.

This design keeps comparison computationally tractable and audit-friendly. Most work is performed through deterministic graph operations, while LLM reasoning is reserved for semantic comparisons that cannot be resolved by exact graph structure alone. The resulting confirmed findings enrich the same knowledge graph used by future ingestion cycles, closing the loop between extraction, validation, storage, and comparison.

5 Evaluation

We evaluate the extraction core of KBExtractor against the MINE-1 knowledge-retention benchmark introduced with KGGen (Mo et al. 2025). MINE-1 measures fact retention and recoverability: for each article, a system builds a knowledge graph, each of the article's 15 known facts is embedded, and the top-$k = 8$ nearest graph nodes plus their two-hop neighborhoods are retrieved. An LLM judge then decides whether each known fact is entailed by the retrieved subgraph. The final MINE-1 score is the mean percentage of known facts captured across articles.

This benchmark is well aligned with our goal because it tests whether graph construction preserves source knowledge in a retrievable form. At the same time, we interpret it carefully. MINE-1 is not a complete measure of graph correctness: a higher fact-retention score does not by itself prove that a graph has fewer hallucinated edges, better legal interpretation, or better downstream compliance utility. It measures whether known facts can be recovered from the graph representation.

5.1 Benchmark Setup and Fairness Controls

We compare KBExtractor against KGGen using the KGGen benchmark, retriever, and metric. To isolate the extraction method rather than the underlying language model, both systems use the same extraction backbone: DeepSeek-R1 32B at temperature 0. We regenerate KGGen graphs using this backbone rather than relying on pre-built graphs from other models, so that both systems are evaluated under the same extraction core, the same all-MiniLM-L6-v2 retriever, the same top-$k = 8$ nearest-node setting, the same two-hop expansion, and the same judge for each run.

The primary evaluation uses a DeepSeek-R1 32B judge on the full 100-article set. We also run a Qwen3-30B-Instruct judge on the same 100 articles as an open-source robustness check. Finally, we include a GPT-5 high-reasoning judge

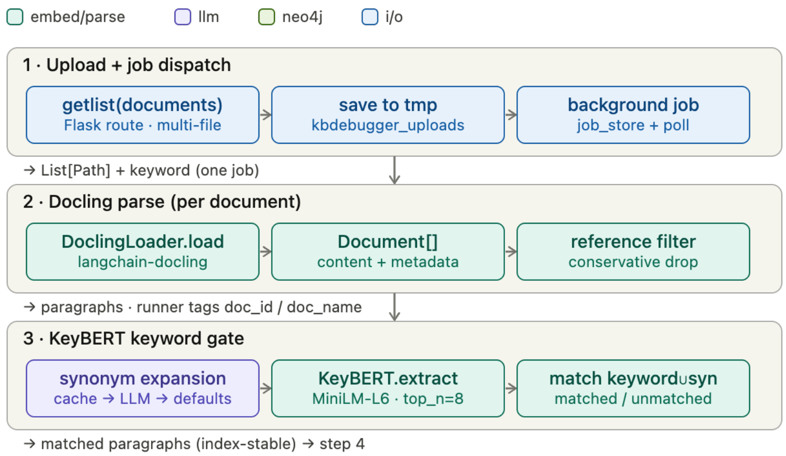

Figure 4: Control points in the ingestion pipeline. The subgraph read, oversight gate, triplet review, and append-only upsert are the main points where context, human judgment, and persistence affect downstream behavior.

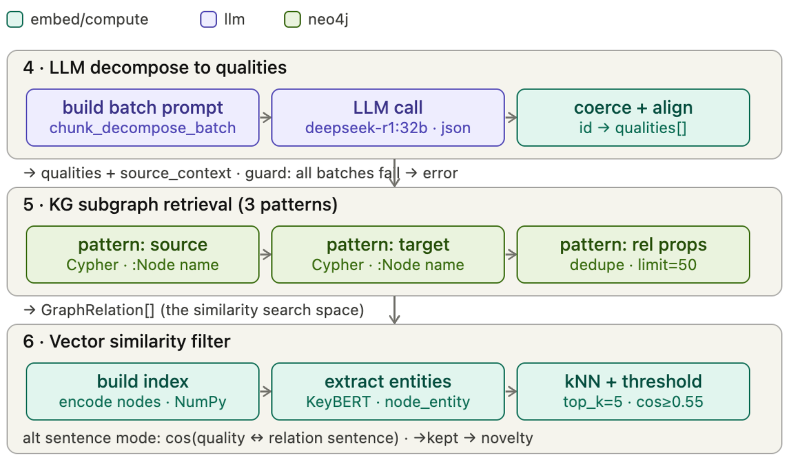

Figure 5: Human review gates used by the methodology. Candidate statements and extracted triplets can be inspected, accepted, rejected, or revised before graph insertion.

ablation on 10 articles. The GPT-5 run is intentionally limited to 10 articles because of evaluation cost, and is therefore used only as a small-scale consistency check rather than as the primary result.

We also report one implementation detail for transparency. KGGen successfully generated 98 of 100 graphs at temperature 0. For two articles, greedy decoding produced object-less relations that were rejected by KGGen's strict schema parser; those two KGGen graphs were therefore regenerated at temperature 0.5. KBExtractor generated all 100 benchmark graphs at temperature 0. The evaluation mode for KBExtractor is also not the complete production user-interface pipeline. It evaluates the extraction core used for MINE, without the full human-review and interactive filtering workflow described earlier.

5.2 Knowledge-Retention Distributions

Figure 13 shows the MINE-1 distribution when DeepSeek-R1 32B is used as the judge. KBExtractor captures an average of 79.4% of expected facts, compared with 62.7% for KGGen. The absolute gain is 16.7 percentage points. KBExtractor also has a narrower distribution (standard deviation 13.5 versus 21.8), suggesting that it is not only stronger on average but also more stable across articles under this judge.

Figure 14 reports the same comparison under Qwen3-30B-Instruct. Both systems receive higher absolute scores under this judge, but the relative ordering remains unchanged. KBExtractor reaches 92.9% mean fact capture, while KGGen reaches 83.7%, yielding a 9.1-point advantage. The standard deviation is again lower for KBExtractor (8.3 versus 15.3), reinforcing the observation that the advantage is not specific to the primary judge.

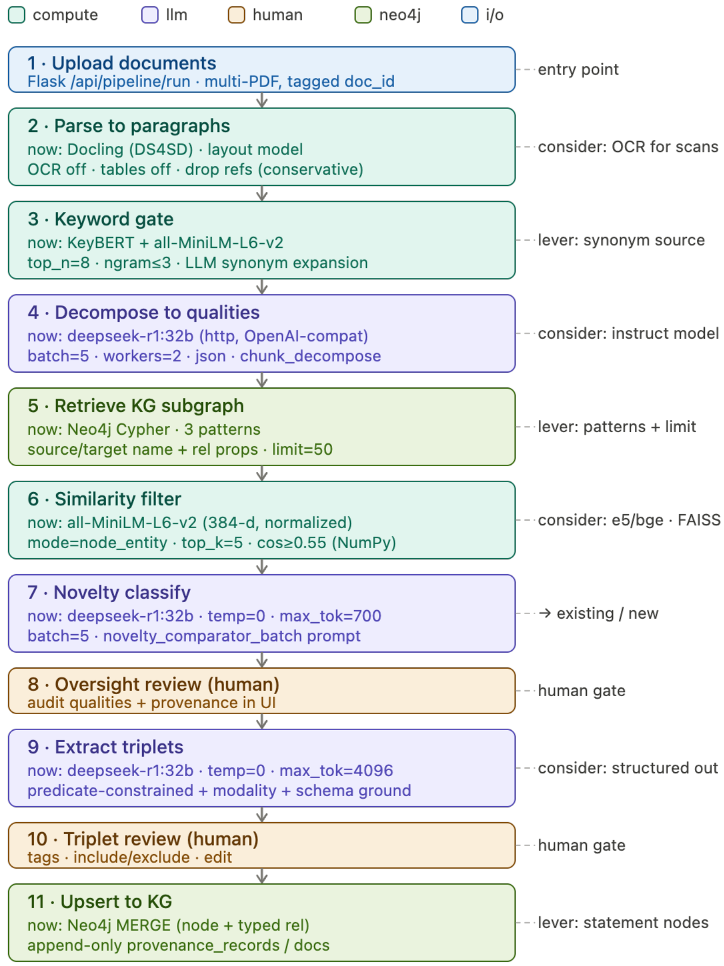

Figure 6: Data shapes emitted by the ingestion stages. Each stage produces an auditable intermediate object, such as paragraph chunks, candidate qualities, neighbor contexts, novelty labels, triplets, validation statuses, and provenance records.

Table 1: MINE-1 benchmark summary. Scores report mean percentage of expected facts captured.

Judge	KBExtractor	KGGen	Gain
DeepSeek-R1 32B ($n = 100$)	79.4%	62.7%	+16.7
GPT-5 high ($n = 10$)	79.3%	52.7%	+26.7
Qwen3-30B ($n = 100$)	92.9%	83.7%	+9.1

5.3 Judge Ablation

Figure 15 summarizes the judge ablation. KBExtractor outperforms KGGen under every judge tested. Under DeepSeek-R1 32B, the mean score improves from 62.7% to 79.4%. Under GPT-5 on the smaller 10-article subset, the score improves from 52.7% to 79.3%, a 26.7-point gain. Under Qwen3-30B-Instruct, the score improves from 83.7% to 92.9%. Absolute score levels vary with judge strictness, but the ranking is consistent.

5.4 Per-Article Paired Comparison

The paired comparisons in Figures 16 and 17 compare KBExtractor and KGGen article by article. Points above the diagonal indicate articles for which KBExtractor captures a larger fraction of the expected facts. Under the DeepSeek-R1 judge, KBExtractor wins on 72 articles, ties on 9, and loses on 19. Under the Qwen3-30B-Instruct judge, KBExtractor wins on 60 articles, ties on 22, and loses on 18. Thus, the aggregate improvement is supported by per-instance dominance rather than by a small number of outlier articles.

5.5 Interpretation and Limitations

The results provide evidence that KBExtractor's extraction core preserves more benchmark facts in a retrievable graph form than KGGen under matched backbone, retriever, and judge conditions. This is a strong result because the comparison uses KGGen's own MINE-1 protocol rather than a custom metric designed only for our system. The main conclusion is therefore about relative fact recoverability: KBEx-

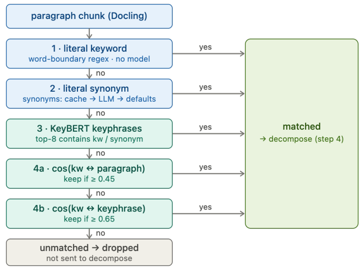

Figure 7: Four-tier keyword gate used before candidate-statement decomposition. Literal keyword and synonym hits are resolved first; embedding-based fallbacks are used only for unresolved paragraphs.

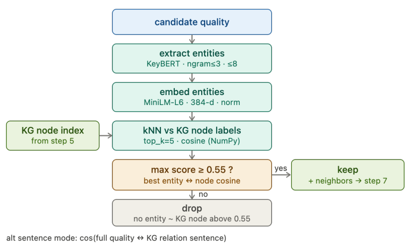

Figure 8: Node-entity similarity filter. Entity-like keyphrases extracted from candidate statements are matched against node labels in the retrieved subgraph; matched neighbor relations become the context for novelty assessment.

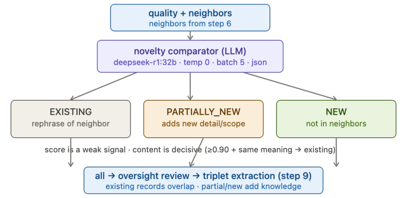

Figure 9: LLM-based novelty decision. Candidate statements are judged against graph-neighbor context and labeled as existing, partially new, or new.

tractor produces graphs from which the benchmark facts are more often recoverable.

Several limitations remain. First, MINE-1 does not measure full graph correctness, legal validity, or hallucinated-edge rate. A system could retain more known facts while still producing additional noisy edges, so fact retention should be complemented with precision-oriented graph-quality evaluation. Second, the GPT-5 judge ablation is only a 10-article check because of cost, while the DeepSeek-R1 and Qwen3 runs cover all 100 articles. Third, two KGGen graphs required temperature 0.5 because temperature 0 produced invalid KGGen outputs. Fourth, the MINE evaluation uses the KBExtractor extraction core rather than the full production workflow with human review and interactive filtering. For these reasons, we present the benchmark as evidence of strong knowledge-retention performance, not as a complete validation of end-to-end compliance correctness.

6 Conclusion

By closing the loop from extraction to mitigation, we reduce the cognitive load on AI developers and ensure that governance requirements are not just “paper promises” but technical realities.

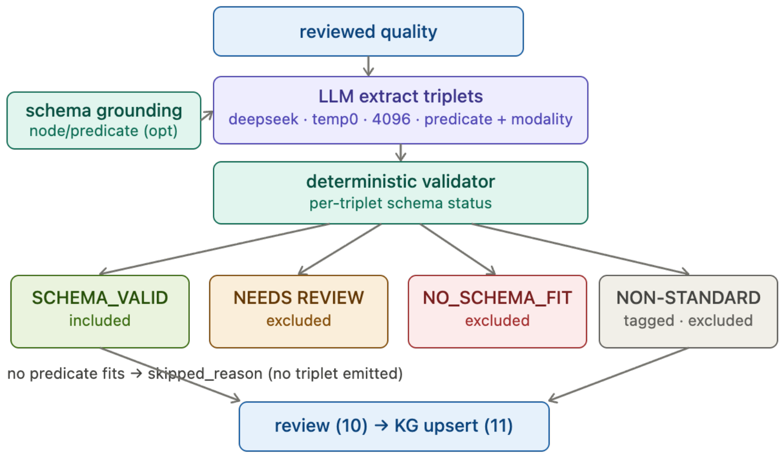

Figure 10: Triplet extraction and deterministic schema validation. The LLM proposes candidate triplets, while source-support and allowed-predicate checks decide whether each triplet is valid, requires review, has no schema fit, or uses a non-standard predicate. Predicate-family grouping is not implemented in the current prototype.

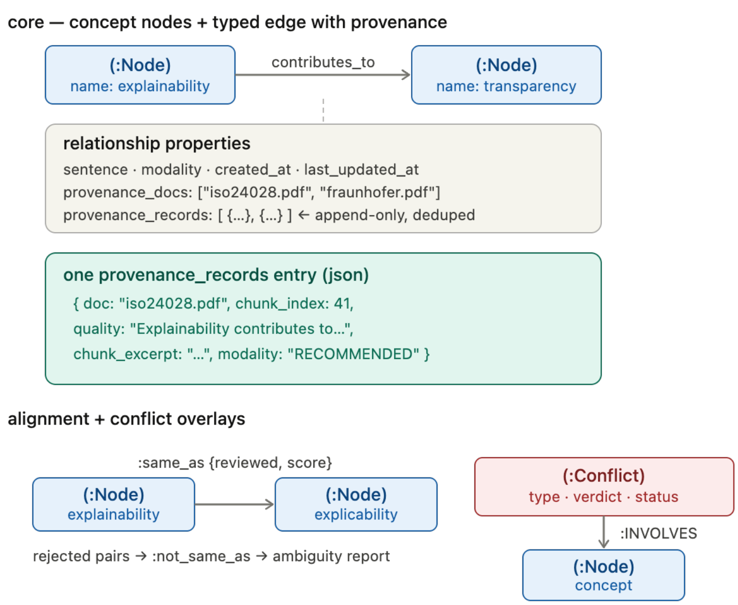

Figure 11: Neo4j data model used for provenance-aware graph storage. Concept nodes are connected by typed relations with append-only provenance records; alignment and conflict overlays represent same-as, not-same-as, and conflict evidence.

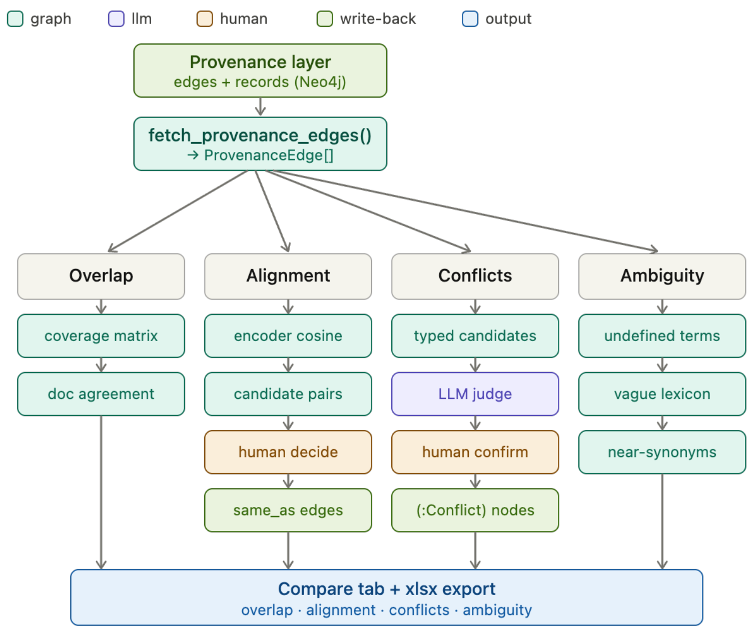

Figure 12: Comparison engine. The engine reads provenance-aware graph data, narrows candidate comparison pairs through graph queries, uses LLM judgment only on candidate pairs, routes decisions through human confirmation, and stores confirmed findings as graph overlays.

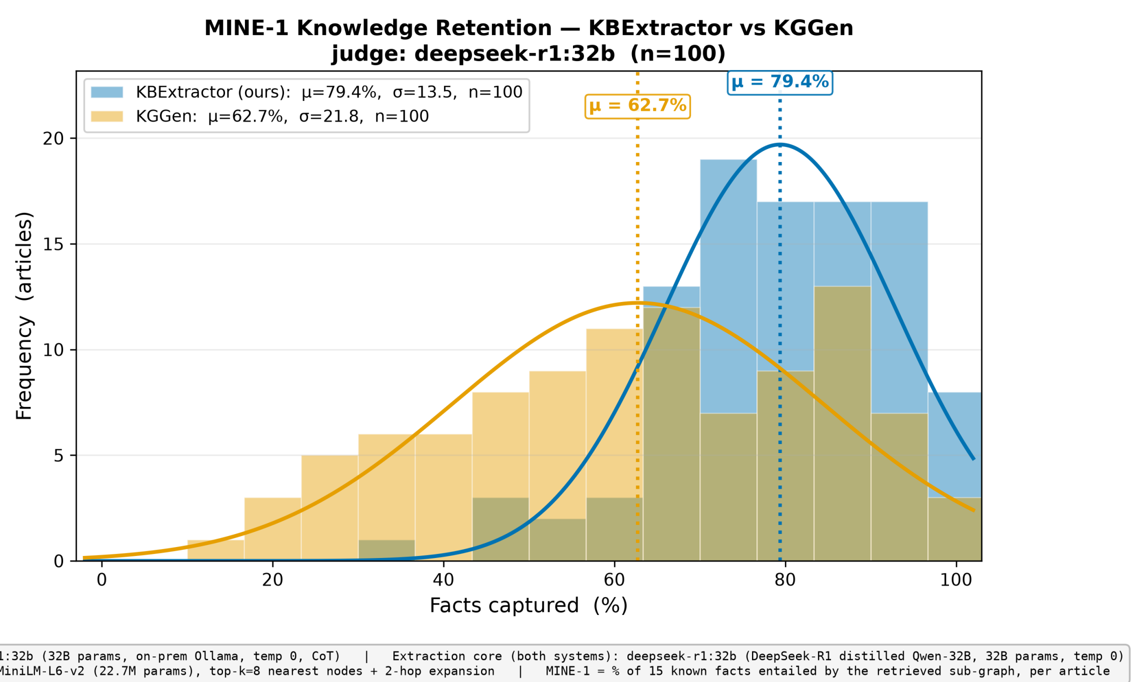

Figure 13: MINE-1 knowledge-retention distribution under the DeepSeek-R1 32B judge. KBExtractor captures more expected facts than KGGen on average and shows lower variance across the 100-article benchmark.

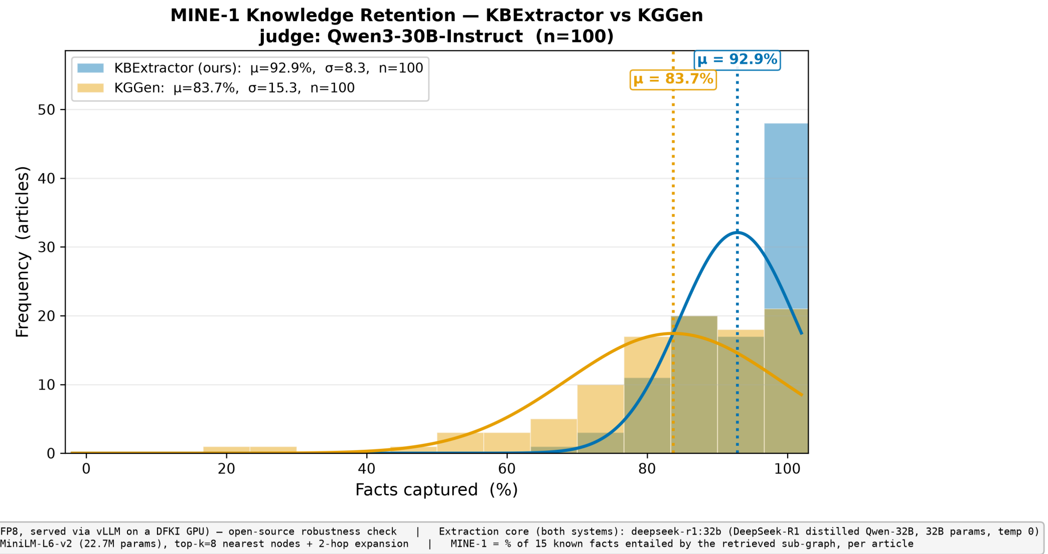

Figure 14: MINE-1 knowledge-retention distribution under the Qwen3-30B-Instruct judge. The absolute scores are higher than under DeepSeek-R1, but KBExtractor retains a clear advantage over KGGen.

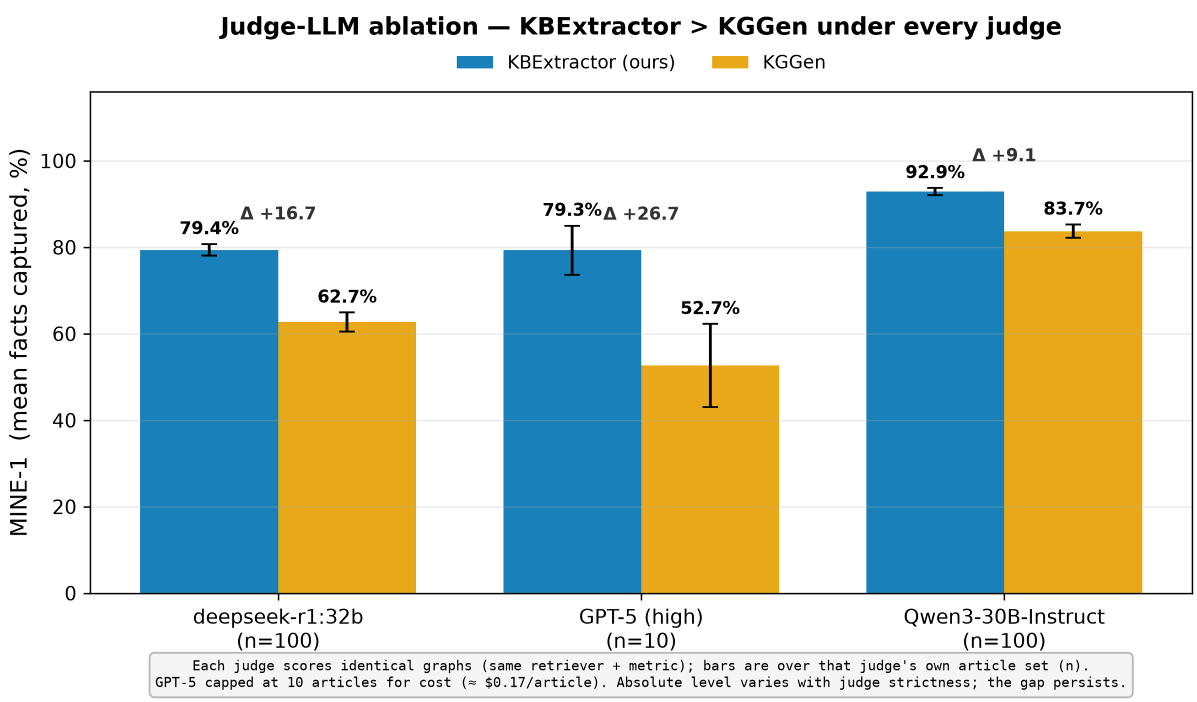

Figure 15: Judge-LLM ablation for the MINE-1 benchmark. KBExtractor outperforms KGGen under DeepSeek-R1 32B, GPT-5, and Qwen3-30B-Instruct judges.

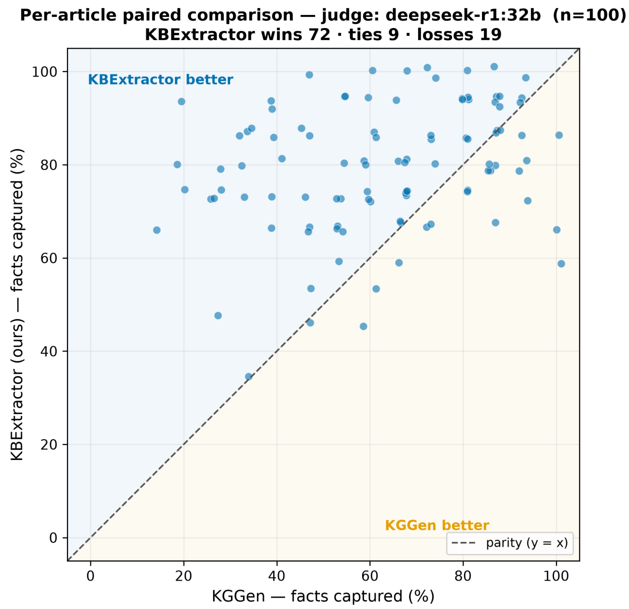

Figure 16: Per-article paired MINE-1 comparison under the DeepSeek-R1 32B judge. KBExtractor wins on 72 of 100 articles, ties on 9, and loses on 19.

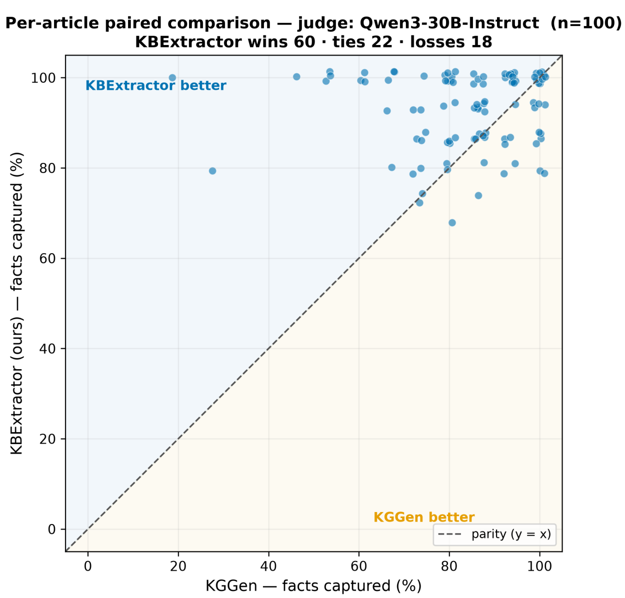

Figure 17: Per-article paired MINE-1 comparison under the Qwen3-30B-Instruct judge. KBExtractor wins on 60 of 100 articles, ties on 22, and loses on 18.

References

Abualhaija, S.; Ceci, M.; Sannier, N.; Bianculli, D.; Lannier, S.; Siclari, M.; Voordeckers, O.; and Tosza, S. 2025. LLM-Assisted Extraction of Regulatory Requirements: A Case Study on the GDPR. In *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, 142–154. IEEE.

Baldwin, W.; and Ghanavati, S. 2026. Knowledge Graph Representations for LLM-Based Policy Compliance Reasoning. *arXiv preprint arXiv:2604.27713*.

Chalkidis, I.; Fergadiotis, M.; Malakasiotis, P.; Aletras, N.; and Androutsopoulos, I. 2020. LEGAL-BERT: The Muppets Straight Out of Law School. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2898–2904.

European Parliament and Council of the European Union. 2024. Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence. https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Guliani, K.; Gill, D.; Landsman, D.; Eshraghi, N.; Kumar, K.; and Gondara, L. 2026. De Jure: Iterative LLM Self-Refinement for Structured Extraction of Regulatory Rules. *arXiv preprint arXiv:2604.02276*.

Hagendorff, T. 2020. The Ethics of AI Ethics: An Evaluation of Guidelines. *Minds and Machines*, 30: 99–120.

Ho-Dac, M. 2024. First Analysis of the EU Artificial Intelligence Act: Towards a Global Standard for Trustworthy AI? *arXiv preprint arXiv:2408.08318*.

Hogan, A.; Blomqvist, E.; Cochez, M.; D'amato, C.; de Melo, G.; Gutierrez, C.; Kirrane, S.; Gayo, J. E. L.; Navigli, R.; Neumaier, S.; Ngomo, A.-C. N.; Polleres, A.; Rashid, S. M.; Rula, A.; Schmelzeisen, L.; Sequeda, J.; Staab, S.; and Zimmermann, A. 2021. Knowledge Graphs. *ACM Computing Surveys*, 54(4): 1–37.

International Organization for Standardization. 2023. ISO/IEC 42001:2023 Information Technology—Artificial Intelligence—Management System. International Standard, https://www.iso.org/standard/81230.html.

Jobin, A.; Ienca, M.; and Vayena, E. 2019. The Global Landscape of AI Ethics Guidelines. *Nature Machine Intelligence*, 1: 389–399.

Lebo, T.; Sahoo, S.; McGuinness, D.; Belhajjame, K.; Cheney, J.; Corsar, D.; Garijo, D.; Soiland-Reyes, S.; Zednik, S.; and Zhao, J. 2013. PROV-O: The PROV Ontology. W3C Recommendation, https://www.w3.org/TR/prov-o/.

Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.

Meroño-Peñuela, A.; Simperl, E.; Kurteva, A.; and Reklos, I. 2025. KG.gov: Knowledge Graphs as the Backbone of Data Governance in AI. *Journal of Web Semantics*, 85: 100847.

Mittelstadt, B. 2019. Principles Alone Cannot Guarantee Ethical AI. *Nature Machine Intelligence*, 1: 501–507.

Mo, B.; Yu, K.; Kazdan, J.; Mpala, P.; Yu, L.; Cundy, C.; Kanatsoulis, C.; and Koyejo, S. 2025. KGGen: Extracting Knowledge Graphs from Plain Text with Language Models. arXiv:2502.09956.

Organisation for Economic Co-operation and Development. 2019. Recommendation of the Council on Artificial Intelligence. OECD/LEGAL/0449, https://legalinstruments.oecd.org/en/instruments/OECD-LEGAL-0449.

Pipitone, N.; and Alami, G. H. 2024. LegalBench-RAG: A Benchmark for Retrieval-Augmented Generation in the Legal Domain. arXiv:2408.10343.

Tabassi, E. 2023. Artificial Intelligence Risk Management Framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology.